CartPole-v1: Double DQN, PPO and SAC Experimental Tuning, Key Insights and Reflections

Group27
Department of Data Science

Abstract

This report focuses on the experimental design, hyperparameter tuning process, and high-level reflections for three reinforcement learning agents (Double DQN, PPO, and discrete SAC) applied to CartPole-v1. Instead of detailing algorithmic derivations, we highlight the core design choices, how they affect learning stability and performance, and what we can learn from systematic hyperparameter adjustments.

1 Interpretation of the Three Algorithms

In this section we keep only conceptual differences and do not include any derivations.

1.1 PPO

- **Type:** on-policy actor–critic, policy gradient with clipping.
- **Core idea:** directly optimize a stochastic policy, but restrict each update to be "not too far" from the old one via a clipping objective.
- **Key knobs:** discount factor γ, clipping range, rollout length, number of epochs per rollout, actor/critic learning rates, entropy coefficient.
- **Intuition:** tends to be stable and robust; the main trade-off is between *update aggressiveness* and *policy stability*.
- **Understanding:** the trust region similar to TRPO is very suitable for this task, since cartpole need to learn policy in a very small step, otherwise the pole is easy to fall down. And the CLIP algorithm guarantee the convergence speed.

1.2 Soft Actor-Critic (Discrete)

- **Type:** off-policy actor–critic with maximum-entropy objective.
- **Core idea:** optimize expected return plus an entropy term, so the policy remains intentionally stochastic; use twin Q-networks for conservative value estimation.
- **Key knobs:** discount factor γ, temperature α, Q/policy learning rates, batch size, target update rate.
- **Intuition:** highly exploratory and robust when tuned well, but more hyperparameters interact in subtle ways.
- **Understanding:** in SAC, there is a discount on low entropy, so it encourages the model to explore but not converge in a local maximum.

Preprint.

2 Core Experimental Ideas and Innovations

In this project, the main "innovations" are in the *experimental methodology and tuning strategy* rather than in the algorithms themselves:

1. **Stage-wise tuning.** For both SAC and PPO, we used a "Step 1 → Step 2 → …" strategy: in each step we changed only a small subset of hyperparameters and diagnosed the learning curves and evaluation scores. This is more informative than performing a large one-shot hyperparameter search, because it exposes the causal effect of each change.

2. **Hypothesis–test loop around key hyperparameters.** For each failure mode or instability, we did not blindly change settings but first formed hypotheses such as:

   - "The learning rate is too large, causing divergence."
   - "The discount factor is too small, making the policy too short-sighted."
   - "The entropy coefficient is too large, preventing convergence."

   Then the next step of tuning served as an experiment to test that hypothesis, gradually building intuition about the *behavior* of each algorithm.

3. **Interpreting algorithms through the shape of their curves.** Instead of only checking whether the final average return is close to 500, we paid attention to:

   - Convergence speed: by which episode do we reach high returns?
   - Collapse patterns: do we see "reach 500 then suddenly drop"?
   - Long-term stability: does performance stay high or keep oscillating?

   These curve shapes reveal bias/variance trade-offs and the effects of the update rules behind each algorithm.

4. **Comparing on-policy vs off-policy robustness.** Although we focus on SAC and PPO tuning here, a core conceptual question is: *On a simple environment like CartPole, is performance mainly limited by sample efficiency or by numerical stability?* The experiments suggest that once samples are plentiful, stability and hyperparameter sensitivity become the dominant issues.

5. **Viewing entropy as a controllable "dimmer switch".** In SAC, a fixed large α keeps the policy high-entropy for a long time: the agent becomes quite "clever but unwilling to commit" to a deterministic strategy. After reducing α, the exploration–exploitation balance matches the task better. This illustrates that entropy regularization is not "the bigger the better"; it must be tuned relative to environment complexity and training stage.

The next two sections document the SAC and PPO tuning processes and the associated reflections.

3 Agent: Double SAR (SAC)

3.1 Hyperparameters

3.2 Step 1

Starting parameters.

- $\gamma = 0.5$, `BATCH_SIZE` $= 32$, `MEMORY_SIZE` $= 50{,}000$, `INITIAL_EXPLORATION_STEPS` $= 1{,}000$.
- `Q_LR` $= 1 \times 10^{-2}$, `PI_LR` $= 1 \times 10^{-2}$, $\alpha = 0.4$, `TARGET_UPDATE_TAU` $= 0.005$, `TARGET_UPDATE_INTERVAL` $= 10$.

Hyperparameter	What it controls
Entropy Coefficient (α)	Balance between reward maximization and policy entropy – the core of SAC's exploration–exploitation tradeoff
Learning Rate (`Q_LR`, `PI_LR`)	How fast the Q-network and policy network weights are updated
Discount Factor (γ)	How much the agent values future rewards vs immediate rewards
Batch Size	Number of transitions sampled from replay buffer for each learning update
Replay Buffer Size	How many past experiences the agent can store and learn from
Target Update Tau (τ)	Soft update coefficient for slowly synchronizing target networks with online network
Target Update Interval	How often (in steps) target network updates are performed
Network Architecture	Width and depth of both Q-networks and policy network
Gradient Clipping Norm	Maximum allowed gradient norm to prevent exploding gradients
Initial Exploration Steps	Number of steps to collect random experiences before starting training
Automatic Entropy Tuning	Whether α is learned automatically to match target entropy
Target Entropy	Desired entropy level for automatic α adjustment (often $-\dim(\mathcal{A})$)

3.2.1 Training progress

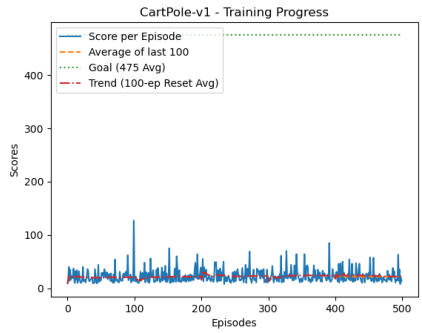

Figure 1: Training progress of Double SAC (Step 1).

Evaluation. [Eval] Average over 100 episodes: 73.45.

Comments. The evaluation score fails to meet the target. The graph shows that the model was not learning in the whole process, with scores oscillating around 30.

Cause analysis.

- $\gamma = 0.5$ too low: over-prioritizing immediate rewards prevents the agent from learning long-horizon strategies essential for CartPole-v1, leading to suboptimal and myopic policies.
- `Q_LR` = `PI_LR` = 10^{-2} too high: excessively large learning rates cause aggressive updates that destabilize training and induce oscillations around optimal policy regions.

3.3 Step 2

Updated parameters.

- $\gamma = 0.9$, `BATCH_SIZE` = 32, `MEMORY_SIZE` = 50,000, `INITIAL_EXPLORATION_STEPS` = 1,000.
- `Q_LR` = 1×10^{-3}, `PI_LR` = 1×10^{-3}, $\alpha = 0.4$, `TARGET_UPDATE_TAU` = 0.005, `TARGET_UPDATE_INTERVAL` = 10.

3.3.1 Training progress

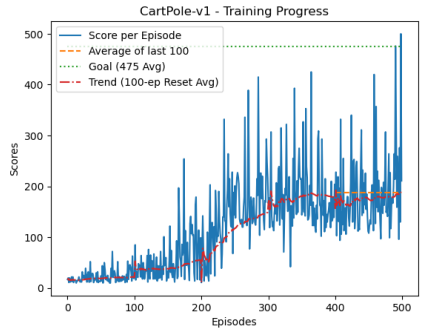

Figure 2: Training progress of Double SAC (Step 2).

Evaluation. [Eval] Average over 100 episodes: 486.51.

Comments. The evaluation score reaches the requirement. The training progress looks better than in Step 1: we can see a learning trend starting from around 200 episodes. However, though the model has reached high scores several times, the policy does not stabilize, only reaching an average score of about 180 in the end.

Cause analysis.

- $\gamma = 0.9$ still too low for long-horizon optimal behavior.
- `Q_LR` = `PI_LR` $= 10^{-3}$ slightly high, still allowing noisy, aggressive updates.
- $\alpha = 0.4$ too high: a fixed, relatively large entropy coefficient enforces excessive stochasticity in the policy, making it difficult to lock in consistent actions and leading to repeated performance collapses after brief peaks.

3.4 Step 3

Updated parameters.

- $\gamma = 0.99$, `BATCH_SIZE` = 32, `MEMORY_SIZE` = 50,000, `INITIAL_EXPLORATION_STEPS` = 1,000.
- `Q_LR` $= 3 \times 10^{-4}$, `PI_LR` $= 3 \times 10^{-4}$, $\alpha = 0.3$, `TARGET_UPDATE_TAU` = 0.005, `TARGET_UPDATE_INTERVAL` = 10.

3.4.1 Training progress

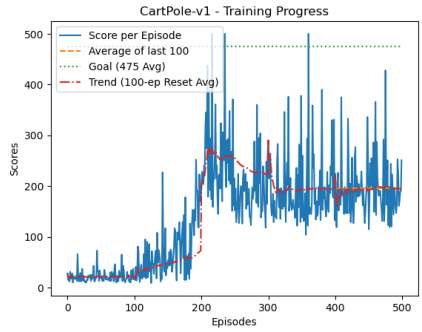

Figure 3: Training progress of Double SAC (Step 3).

Evaluation. [Eval] Average over 100 episodes: 449.06.

Comments. The evaluation score does not meet the requirement. We can see a sharp rise of performance around 200 episodes, reaching 500, which is better than Step 2. However, the performance stops improving and even drops during the last 300 episodes. The severe oscillation problem is slightly reduced but still present.

Cause analysis.

- `BATCH_SIZE` = 32 too small: limited batch size increases gradient noise in Q-function updates during later training, hindering fine-tuning and contributing to performance regression.
- `TARGET_UPDATE_INTERVAL` = 10 suboptimal: updating the target network only every 10 steps—despite soft updates—introduces lag in target alignment, allowing value overestimation to accumulate and destabilize a learned policy.

3.5 Step 4

Updated parameters.

- $\gamma = 0.99$, `BATCH_SIZE` = 64, `MEMORY_SIZE` = 50,000, `INITIAL_EXPLORATION_STEPS` = 1,000.
- `Q_LR` = 3×10^{-4}, `PI_LR` = 3×10^{-4}, $\alpha = 0.3$, `TARGET_UPDATE_TAU` = 0.005, `TARGET_UPDATE_INTERVAL` = 1.

3.5.1 Training progress

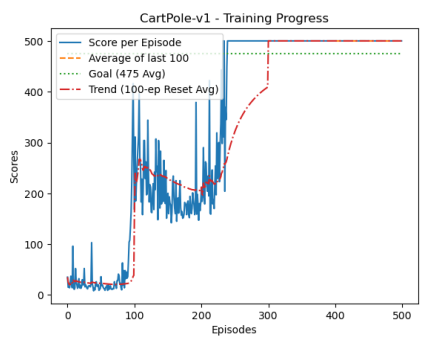

Figure 4: Training progress of Double SAC (Step 4).

Evaluation. [Eval] Average over 100 episodes: 500.00.

Comments. The evaluation score reaches the requirement. The learning process reaches perfect performance without oscillation after about 250 episodes.

Reflection. From Step 1 to Step 4, we can clearly see:

- The discount factor γ controls whether the policy is "short-sighted" or willing to sacrifice immediate reward for long-term balance.
- The learning rate and batch size determine the noise level in Q-updates.
- The entropy coefficient α controls the balance between "always exploring" and "starting to commit".

The final successful setting is essentially where "high-entropy exploration" and "stable value estimation" both work properly at the same time.

4 Agent: PPO

4.1 Hyperparameters

Hyperparameter	What it controls
`ACTOR_LR`	Learning rate of the policy network
`CRITIC_LR`	Learning rate of the value function
`GAMMA`	Discount factor for returns
`GAE_LAMBDA`	Bias–variance trade-off in advantage estimation (λ in GAE-λ)
`CLIP_EPSILON`	PPO clipping range – limits how far the new policy can move away from the old one
`ROLLOUT_LENGTH`	How many steps are collected before each PPO update (n_{step} in literature)
`PPO_EPOCHS`	How many times we reuse each rollout (K in the PPO paper)
`BATCH_SIZE`	Mini-batch size inside each PPO epoch
`ENTROPY_COEF`	Strength of entropy bonus (encourages exploration)
`VALUE_COEF`	Weight of value-function loss
`MAX_GRAD_NORM`	Gradient clipping threshold

4.2 Step 1

Starting parameters.

- `GAMMA` $= 0.9$, `GAE_LAMBDA` $= 0.95$, `CLIP_EPSILON` $= 0.3$, `ROLLOUT_LENGTH` $= 512$, `PPO_EPOCHS` $= 4$, `BATCH_SIZE` $= 128$.
- `ACTOR_LR` $= 3 \times 10^{-3}$, `CRITIC_LR` $= 1 \times 10^{-2}$, `ENTROPY_COEF` $= 0.01$, `VALUE_COEF` $= 0.5$, `MAX_GRAD_NORM` $= 0.5$.

4.2.1 Training progress

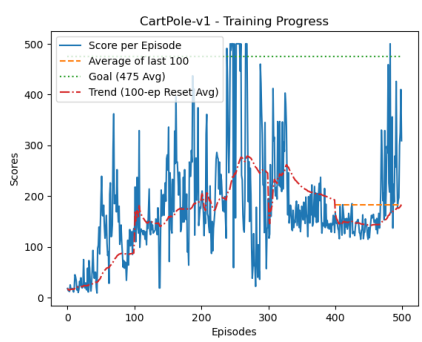

Figure 5: Training progress of PPO (Step 1).

Evaluation. [Eval] Average over 100 episodes: 447.27.

Comments. Evaluation scores do not meet the requirement. During episodes 0–300, scores climb steeply to the maximum 500. After reaching peak performance, there's an abrupt drop in scores, indicating the policy has been destabilized by aggressive updates. During episodes 300–500, scores remain volatile with no recovery to the optimal 500, showing the algorithm cannot maintain a stable optimal policy.

Cause analysis.

- `GAMMA`$= 0.9$ too low: discounts future rewards too aggressively, so the policy becomes short-sighted and fails to maintain long-term balance.
- `CRITIC_LR`$= 10^{-2}$ excessively high: allows too large policy/value updates per step; later-stage fine-tuning becomes destructive noise rather than improvement.

- CLIP_EPSILON= 0.3 overly permissive: large clipping range enables dramatic policy shifts; the delicate optimal balancing policy gets overwritten by random exploration.

4.3 Step 2

Updated parameters.

- GAMMA = 0.99, GAE_LAMBDA = 0.95, CLIP_EPSILON = 0.15, ROLLOUT_LENGTH = 512, PPO_EPOCHS = 4, BATCH_SIZE = 128.
- ACTOR_LR = 3×10^{-3}, CRITIC_LR = 1×10^{-3}, ENTROPY_COEF = 0.01, VALUE_COEF = 0.5, MAX_GRAD_NORM = 0.5.

4.3.1 Training progress

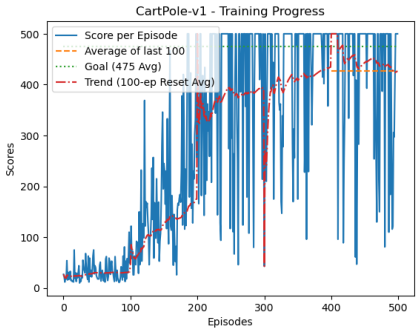

Figure 6: Training progress of PPO (Step 2).

Evaluation. [Eval] Average over 100 episodes: 500.00.

Comments. The agent achieves 500 scores within 200 episodes, demonstrating successful early learning. However, performance collapses abruptly after reaching the peak and remains volatile for the remainder of training, failing to stabilize at the optimal policy. This indicates aggressive updates continue to disrupt the learned policy instead of fine-tuning it.

Cause analysis.

- ROLLOUT_LENGTH= 512 too long: accumulates highly correlated samples from CartPole's limited state space, which amplifies variance in gradient estimates and destabilizes updates.
- PPO_EPOCHS= 4 excessive for a simple environment: repeated optimization on the same batch leads to overfitting and policy oscillation around the optimum.
- ACTOR_LR= 3×10^{-3} too high: large updates in later training overwrite the delicate balancing policy with noisy gradients.
- CLIP_EPSILON= 0.15 remains too permissive: allows policy shifts large enough to break the finely tuned balance strategy.

4.4 Step 3

Updated parameters.

- GAMMA = 0.99, GAE_LAMBDA = 0.95, CLIP_EPSILON = 0.1, ROLLOUT_LENGTH = 256, PPO_EPOCHS = 3, BATCH_SIZE = 64.
- ACTOR_LR = 3×10^{-4}, CRITIC_LR = 3×10^{-4}, ENTROPY_COEF = 0.01, VALUE_COEF = 0.5, MAX_GRAD_NORM = 0.5.

4.4.1 Training progress

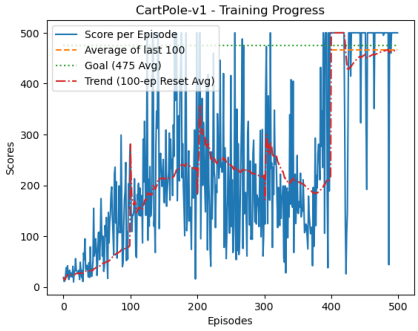

Figure 7: Training progress of PPO (Step 3).

Comments. The agent achieves near-perfect scores around 500 within 200 episodes and maintains stable performance throughout episodes 400–500. The final evaluation confirms complete mastery of the environment with consistent 500 scores, showing the policy has successfully learned and stabilized. The sudden drop of the scores in the training process is caused by exploration steps, which is not an abnormal phenomenon.

Reflection. Compared with SAC tuning, PPO shows a characteristic pattern:

- If the rollout length is too long, the number of PPO epochs too high, and the learning rate too large, the agent often exhibits a "learn well then self-destruct" behavior.
- Reducing `CLIP_EPSILON` and learning rates, and shortening the rollout, makes each policy update more conservative and allows PPO's stability advantages to show up.

5 Agent: From Online to Offline (Offline RL & Imitation)

While PPO and SAC achieved mastery in the online setting, real-world applications often restrict agents to learning from fixed, historical datasets without environment interaction. In this section, we extend our study to the **Offline RL** setting. We explore whether advanced algorithms can recover optimal policies from sub-optimal data, comparing **Behavior Cloning (BC)**, **Advantage-Weighted BC (AWBC)**, and **Conservative Q-Learning (CQL)**.

5.1 Experimental Setup & Data Generation

Instead of assuming perfect demonstrations, we constructed datasets with varying quality to stress-test the algorithms, inspired by robustness studies:

- **D-Expert:** 50k transitions collected by our converged SAC agent (Step 4 model).
- **D-Mixed (The Challenge):** 50k transitions collected with a noise injection rate of $p_{rand} = 0.5$. This dataset mimics a "human-level" replay buffer containing both successful trajectories and random failures.

5.2 Step 1: Baseline Verification on Expert Data

Hypothesis. On `D-Expert`, simple imitation (BC) should suffice and converge fastest. CQL should also succeed but may train slower due to the dual-network (Actor–Critic) complexity.

Parameters.

- **BC:** `LR` $= 3 \times 10^{-4}$, `BATCH_SIZE` $= 256$, `EPOCHS` $= 50$.

- **AWBC:** LR $= 3 \times 10^{-4}$, BATCH_SIZE $= 256$, EPOCHS $= 50$ BETA $= 2$ W_CLIP $= 20$.
- **CQL:** LR $= 3 \times 10^{-4}$, BATCH_SIZE $= 256$ GAMMA $= 0.99$, ALPHA_CQL $= 2.0$, TEMPERATURE $= 1.0$, NUM_RANDOM_ACTIONS $= 10$, ENTROPY_ALPHA $= 0.05$, TARGET_UPDATE_INTERVAL $= 1$.

5.2.1 Training progress

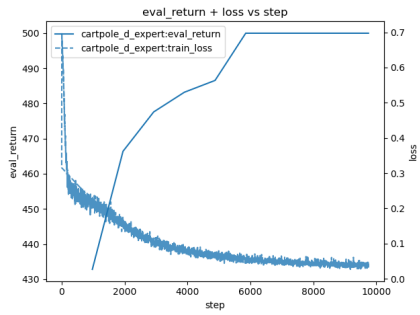

Figure 8: Training progress of BC.

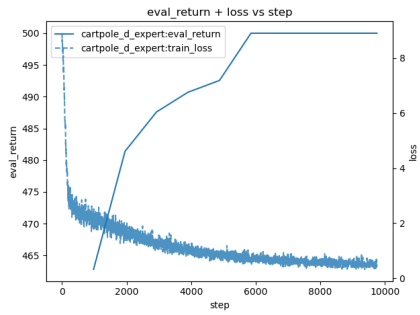

Figure 9: Training progress of AWBC.

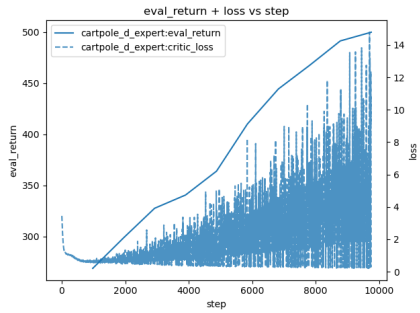

Figure 10: Training progress of CQL.

Observation & Analysis. Both BC and AWBC achieved an average evaluation score of **500.0**. BC and AWBC converged within about 6000 steps, acting as a perfect supervised learner. CQL took longer (≈ 10000 episodes) to align its Q-values but eventually matched the expert. This confirms our implementations are correct and sets a baseline.

5.3 Step 2: The "Stitching" Test on Mixed Data

Motivation. This is the core experiment. On `D-Mixed`, we will test two cases: the dataset contains 50% random actions and 80% random actions. A naive agent might mimic the failures. We test if algorithms can "filter" or "stitch" good parts.

Comparators.

1. **BC:** Naive supervised learning.
2. **AWBC (Strong Baseline):** Weights updates by normalized return $w(G)$, filtering out low-performing trajectories.
3. **CQL (Frontier):** Penalizes Q-values for OOD actions to perform conservative planning.

5.3.1 50% random actions

Training Progress.

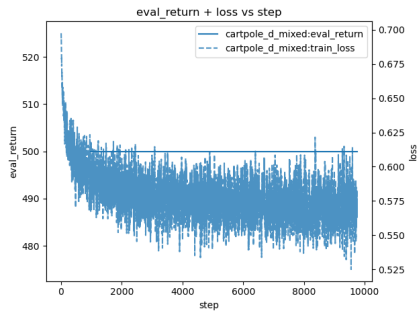

Figure 11: Training progress of BC.

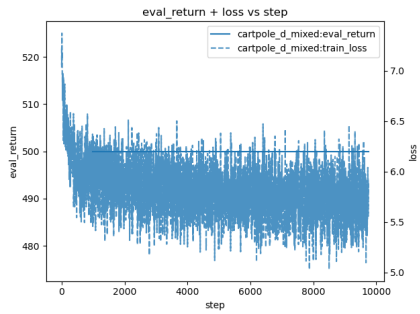

Figure 12: Training progress of AWBC.

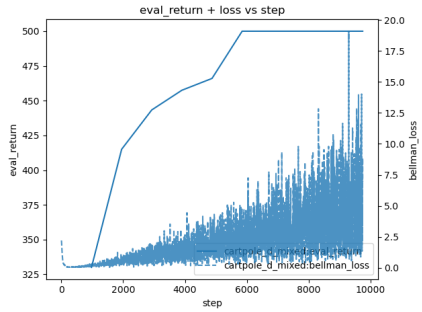

Figure 13: Training progress of CQL.

- **BC:** reaches 500 very quick.
- **AWBC:** reaches 500.
- **CQL:** reaches 500 after a few episodes.

Cause Analysis (The "Stitching" Property).

- **BC and AWBC's quick convergence:** reaches 500 very quickly because the task's optimal policy is low-complexity (a near-linear decision boundary), and the dataset still contains a strong, consistent supervision signal from the expert-labeled transitions—especially in the state regions that matter for long-horizon success. Although half of the actions are random, many random-action segments terminate early and thus contribute mostly short, low-quality trajectories; as a result, the effective supervision on the "successful" state distribution is dominated by expert actions, allowing BC to fit a near-optimal mapping rapidly and saturate at the episode cap.
- **CQL's slow convergence:** CQL converges more slowly because it must first learn a self-consistent value function through bootstrapping while simultaneously applying a conservative regularizer that suppresses overestimation on out-of-distribution actions. This conservatism reduces early policy-improvement gradients and delays performance gains until the critic becomes sufficiently accurate within the dataset support, after which the policy improves and the method converges.

5.3.2 87% random actions

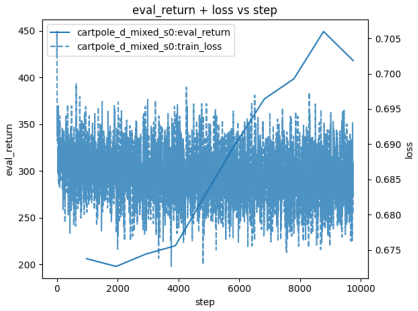

Figure 14: Training progress of BC.

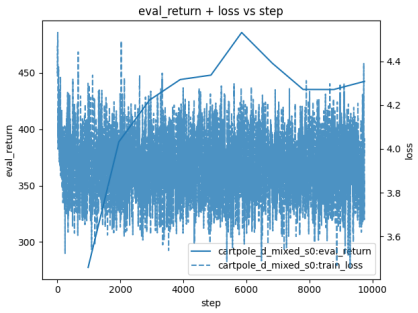

Figure 15: Training progress of AWBC.

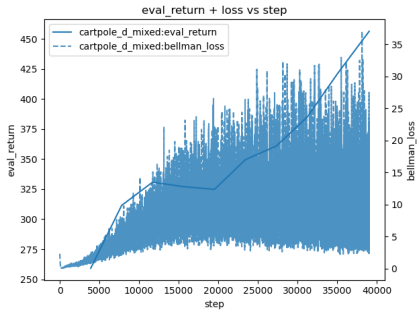

Figure 16: Training progress of CQL($alpha_cql = 1.$)

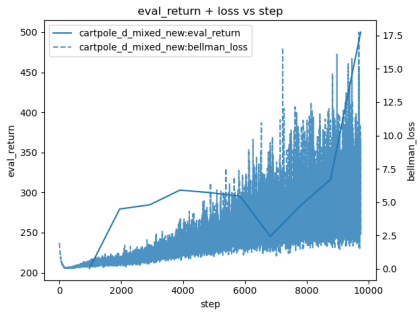

Figure 17: Training progress of CQL($alpha_cql = 2.$)

Training Progress.

- **BC:** fails to be convergent.
- **AWBC:** reaches a return of 450 but not enough.
- **CQL:** if alpha_cql=1, it fails to reach 500. if alpha_cql=2, reaches 500 after about 10000 episodes.

Cause Analysis (The "Stitching" Property).

- **BC and AWBC's failure:** the actions is too random, so the learned policy is pulled toward producing near-random actions as well. Moreover, the dataset contains far fewer long, high-quality trajectories, so BC receives insufficient consistent signal about which actions are necessary to keep the pole balanced; small mistakes then compound over time at evaluation, leading to persistently low returns.
- **CQL's success:** CQL is not trying to imitate the average behavior. Instead, it learns a critic via Bellman backups, which propagate long-horizon value information from the relatively rare but informative successful segments through preceding transitions. Even if good actions appear infrequently, once the critic assigns higher value to transitions that lead into those successful regions, the policy can improve by selecting actions with higher estimated return. Crucially, the conservative regularization in CQL suppresses overestimated values for out-of-distribution actions, preventing the policy from exploiting spurious Q-errors and enabling stable, incremental improvement until success.
  Increasing α_{CQL} from 1.0 to 2.0 was crucial. The setting $\alpha_{CQL} = 1.0$ proved insufficient for the highly noisy (87% random) dataset, failing to suppress overestimation of OOD actions. By increasing the penalty to $\alpha_{CQL} = 2.0$, the algorithm became more "pessimistic," effectively filtering out the noise and allowing the underlying optimal structure to emerge. In offline RL, the Q-function is only well-supervised on in-dataset state–action pairs; for out-of-distribution (OOD) actions it must extrapolate, which often produces spuriously high values. The policy update then tends to exploit these inflated estimates, selecting

OOD actions that look good to the critic but perform poorly in the real environment, leading to unstable or suboptimal learning.

5.4 Step 3: Robustness to OOD States

Hypothesis. Pure imitation learning (BC/AWBC) memorizes the dataset's state distribution. RL-based methods (CQL) learn the underlying physics (dynamics) via the Bellman equation, offering better generalization.

Experiment. Evaluation Results.

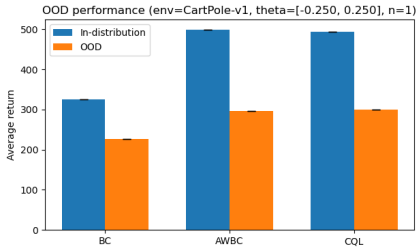

Figure 18: OOD OF $\theta = 0.25$

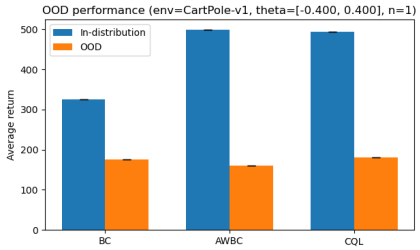

Figure 19: OOD OF $\theta = 0.40$

- **BC/AWBC:** The average OOD return of BC is 226.55. The average OOD return of AWBC is 297.00.
- **CQL:** The average OOD return of CQL is 300.60.

Reflection. CQL achieves better OOD performance because its learned policy is driven by an explicit estimate of long-horizon value rather than direct action imitation. Under distribution shift (e.g., larger reset perturbations), BC-style methods often encounter states that are weakly represented in the offline dataset, so their predictions can be unreliable and errors compound during rollout. In contrast, CQL learns a critic through Bellman backups, which can generalize value structure across nearby states and provide a directional signal for selecting actions that recover toward high-return regions. Moreover, CQL's conservative regularization suppresses overestimation on out-of-distribution actions, reducing the chance that the policy exploits spurious high-Q values caused by extrapolation. This "pessimism" makes policy improvement more stable under state-distribution shifts and typically yields higher success rates and returns in OOD evaluations compared to imitation-based baselines.

6 Comparative Analysis and Overall Reflections

Combining the Online (DQN/PPO/SAC) and Offline (BC/CQL) experiments, we derive a holistic view, highlighting the core distinctions and learning properties of each paradigm.

6.1 Core Limitations

- **Online Strategies:** Primarily limited by **sample efficiency** and **numerical stability**. Once samples are plentiful, the dominant issue becomes preventing policy collapse through hyperparameter tuning (e.g., entropy in SAC, clipping range in PPO).
- **Offline Strategies:** Primarily limited by **data coverage** and **data quality**. Algorithms must trade off between "trusting the data" and "generalization," and cannot correct erroneous value estimates through environment interaction.

6.2 Learning Mechanism & "Stitching" Capability

- **Online/Imitation Learning (BC/AWBC):** BC and AWBC tend to mimic the average behavior of the dataset or filter out poor trajectories. In the presence of extreme noise (e.g., 87% random actions), they fail due to the lack of a consistent supervision signal, exhibiting "imitation failure." AWBC functions merely as a sophisticated filtering mechanism.
- **Offline RL (CQL):** CQL is not just imitation; it possesses **"Stitching"** capability. Through dynamic programming (Bellman backups), it connects sparse successful segments to generate a new optimal policy from sub-optimal data. CQL employs conservative regularization to suppress value overestimation for OOD actions, enabling stable policy improvement even with incomplete data coverage.

6.3 Tuning Essence & Trade-offs

- **Online Strategies (SAC/PPO):** The essence of tuning is managing the balance between **exploration and commitment**. For instance, the entropy coefficient α in SAC controls whether the agent maintains random exploration or locks into a specific strategy.
- **Offline Strategies (CQL):** The essence of tuning is managing the balance between **conservatism and generalization**. The conservative weight α_{CQL} is the critical knob, balancing "trusting the data" (avoiding OOD overestimation) against "trusting the value function" (extrapolation).

6.4 Robustness

- **Result:** When facing environmental perturbations (such as OOD states), RL-based methods (CQL) outperform pure imitation methods (BC/AWBC).
- **Reason:** This is because CQL learns the underlying value structure and dynamics, rather than merely memorizing state-action mappings.